Implementação e Avaliação de Desempenho de uma Topologia Fat-Tree Compacta no Simulador gem5

Lucas Lali Kusumoto, RA143690

Departamento de Informática – Universidade Estadual de Maringá (UEM)

Abstract

À medida que o número de núcleos em processadores aumenta, a forma como eles se comunicam passa a ditar o desempenho do sistema. Barramentos compartilhados tradicionais não suportam muitos núcleos conectados ao mesmo tempo, exigindo redes internas dedicadas. Este trabalho apresenta a modelagem e a avaliação de uma topologia em árvore hierárquica (Fat-Tree) no subsistema Ruby do simulador gem5. A proposta foca em configurações de 4, 8 e 16 controladores de cache e memória, duplicando a capacidade dos canais próximos à raiz para evitar congestionamentos. Avalia-se o comportamento da Fat-Tree frente às topologias Anel e Mesh sob tráfego sintético e durante a execução de um programa concorrente em C com o processador O3.

1 Introdução

Colocar vários núcleos em um mesmo chip criou um grande problema: como ligar todos eles à memória sem criar filas de espera. O modelo de barramento único, onde todos os núcleos dividem a mesma pipeline, pode gerar perda de tempo quando o sistema tem mais que poucos núcleos [1]. Por causa disso, processadores modernos usam redes internas formadas por pequenos roteadores e canais de comunicação dedicados.

Existem desenhos de rede já bem estabelecidos, mas cada um gera um custo. Uma rede em anel (*Ring*), por exemplo, é simples de ligar e gasta poucos fios, só que uma mensagem precisa passar por quase todos os nós intermediários para chegar ao destino oposto. Já uma malha quadrada (*Mesh*) distribui melhor as conexões em uma grade, mas as trocas de dados distantes continuam lentas, e os roteadores do meio da grade costumam concentrar a maior parte do tráfego, se tornando gargalos.

A alternativa que será estudada nesse trabalho é a *Fat-Tree* proposta por Leiserson [2]. Em uma árvore comum, conforme se sobe em direção ao topo (raiz), todos os nós de baixo disputam as mesmas conexões de cima, gerando filas. A ideia central da Fat-Tree é deixar as conexões mais largas (com maior capacidade de transmissão ou canais duplicados) nos níveis superiores. Assim, quando vários processadores tentam se comunicar ao mesmo tempo com o outro lado da árvore, a rede pode suportar esse volume e evitar gargalos.

O objetivo deste projeto é implementar uma versão simples dessa Fat-Tree no gem5 [3]. Usando o subsistema de memória Ruby com a rede detalhada Garnet [4] para poder coletar os dados requiridos na descrição da atividade, constrói-se a estrutura em árvore para grupos de 4, 8 e 16 controladores de memória e cache. A Fat-Tree implementada

terá o seu desempenho comparado com as opções em Anel e Mesh, a fim de descobrir se o seu ganho de velocidade compensa o custo de colocar mais roteadores e conexões no circuito.

2 Metodologia

2.1 Visão Geral do Projeto

O trabalho é dividido em três etapas sequenciais: a preparação das cargas de teste, a execução da simulação com a nova rede configurada no gem5, e a leitura automatizada dos relatórios gerados.

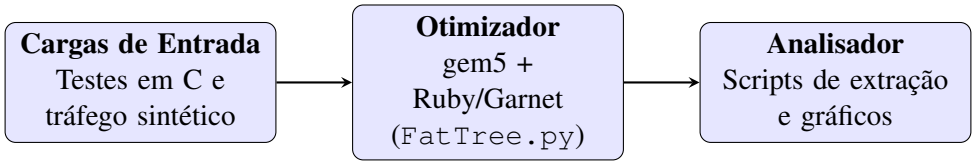


Figura 1: Fluxo de trabalho: da entrada de testes à coleta de resultados.

2.2 Organização dos Módulos

A implementação é dividida em duas partes centrais:

2.2.1 Otimizador: Construção da Fat-Tree

O código principal fica no arquivo `configs/topologies/FatTree.py`. Utiliza-se a estrutura padrão do gem5 para declarar os componentes e criar as conexões da seguinte forma:

- **Folhas (base):** Roteadores que se conectam diretamente aos controladores de cache e memória dos núcleos por meio de conexões externas (`ExtLink`).
- **Níveis superiores (tronco e raiz):** Roteadores intermediários e centrais conectados entre si por enlaces internos (`IntLink`).

Nos enlaces dos níveis mais altos, dobra-se o parâmetro de largura de banda (`bandwidth_factor = 2` ou `4`). Com isso, o canal do topo consegue transmitir duas a quatro vezes mais dados por ciclo de clock do que os canais das pontas. A Figura 2 mostra o esquema básico montado para 8 controladores.

2.2.2 Analisador: Processamento dos Dados

O analisador reúne scripts em Python responsáveis por rodar as simulações em sequência e ler o arquivo de saída gerado pelo gem5 (`m5out/stats.txt`). Esses scripts filtram os dados brutos de tempo, latência de entrega e uso de canais, organizando as informações em tabelas para facilitar a comparação entre as topologias.

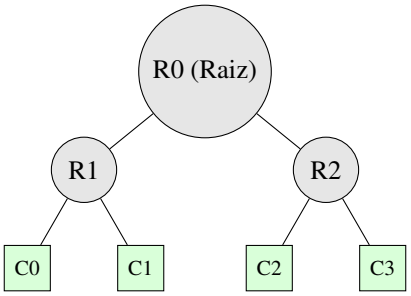


Figura 2: Estrutura básica da Fat-Tree: os controladores ficam na base e as conexões engrossam na direção do roteador raiz.

2.3 Configuração da Arquitetura

Para que os testes reflitam o comportamento de um processador multicore real na busca de dados, adota-se a CPU `X86O3CPU`, que executa instruções fora de ordem (*Out-of-Order*). A Tabela 1 resume os parâmetros que serão adotados no simulador.

Tabela 1: Parâmetros de configuração no gem5.

Parâmetro	**Valor Escolhido**
Modelo de Processador	X86O3CPU (Fora de ordem, 8 instruções por ciclo)
Frequência	2.0 GHz
Caches Privadas L1	Instrução e Dados (32 kB cada, 8 vias)
Protocolo de Coerência	MESI_Two_Level (Ruby)
Modelo de Rede	Garnet com temporização detalhada
Topologias Testadas	FatTree, Mesh e Anel
Escalas Avaliadas	4, 8 e 16 controladores
Largura dos Canais na Base	16 bytes por ciclo de clock
Largura dos Canais no Topo	32 ou 64 bytes por ciclo de clock

2.4 Programas e Cargas Utilizadas

A avaliação do projeto utiliza dois tipos de testes:

1. **Tráfego Sintético de Rede:** Utiliza-se o gerador de tráfego do próprio Garnet para injetar pacotes aleatórios entre os nós com taxas crescentes (de 0.02 a 0.50 pacotes por ciclo). Isso permite verificar até onde cada rede suporta a carga antes de começar a formar filas longas.

2. **Microbenchmark Concorrente em C:** Um programa em C escrito com a biblioteca `pthreads`. O código cria várias tarefas que tentam alterar os mesmos blocos de memória concorrentemente, protegidos por travas de exclusão mútua. Essa dinâmica força as caches a trocarem mensagens constantes.

2.5 Métricas de Avaliação

A análise dos resultados divide-se em três aspectos práticos:

- **Comportamento da Rede:** Tempo médio de entrega dos pacotes (`average_packet_latency`) e taxa de ocupação dos canais internos.
- **Desempenho da Aplicação:** Total de ciclos gastos para terminar o programa em C e taxa média de instruções executadas por ciclo (IPC).
- **Custo de Implementação:** Quantidade total de roteadores e conexões criadas na Fat-Tree em comparação com o Anel e a Mesh, além do tempo que o computador demorou para rodar a simulação.

3 Resultados Esperados

Espera-se que a Fat-Tree compacta consiga sustentar um volume maior de tráfego antes de entrar em sobrecarga, principalmente porque seus canais superiores foram pensados para dar vazão às mensagens que cruzam lados diferentes da árvore. Em comparação, o Anel deve acumular atrasos expressivos nas distâncias maiores, enquanto a Mesh deve sofrer com filas nos roteadores centrais da grade.

No teste com o programa em C e o processador O3, a expectativa é de que tempos menores de resposta na rede evitem que o processador passe ciclos esperando por dados da memória compartilhada. Por fim, espera-se constatar que, em sistemas pequenos com apenas 4 nós, a Fat-Tree adiciona roteadores demais sem trazer grandes vantagens frente ao Anel; contudo, ao subir para 16 nós, a redução nos atrasos de entrega deve compensar a complexidade extra de conexões.

Referências

[1] W. J. Dally and B. P. Towles, *Principles and practices of interconnection networks*. Morgan Kaufmann, 2004.

[2] C. E. Leiserson, "Fat-trees: universal networks for hardware-efficient supercomputing," *IEEE Transactions on Computers*, vol. 100, no. 10, pp. 892–901, 1985.

[3] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Somogyi, *et al.*, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, 2011.

[4] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "Garnet: A detailed on-chip network model inside a full-system simulator," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pp. 33–42, IEEE, 2009.